The Incentive–Learning Frontier: Truthful Reinforcement-Learning PRB Allocation Under Hard Service Floors in O-RAN Slicing

Prashant Mishra
G. S. Sanyal School of Telecommunications
Indian Institute of Technology Kharagpur
Kharagpur, India

Dibbendu Roy
G. S. Sanyal School of Telecommunications
Indian Institute of Technology Kharagpur
Kharagpur, India

Abstract—Open RAN increasingly delegates slice-level physical resource block (PRB) allocation to learning agents in the near-real-time RIC, yet these agents assume the controller observes tenant demand directly. We study the setting in which tenants are strategic and the allocator learns from their reports, so that each report is simultaneously an incentive object and a training sample. We show that exact dominant-strategy truthfulness within a fixed-policy epoch coexists with an unavoidable episode-level incentive slack whenever hard per-slice service floors are active, and we localize that slack precisely to the slots on which a floor binds. Our main result characterizes the achievable tradeoff between cumulative learning regret and incentive slack as a frontier governed by two parameters, the epoch length L and the service-floor binding frequency ρ, and proves that ρ is an exogenous quantity fixed by admission-control load rather than by the learning horizon. The combined cost is of order $\rho^{1/3}T^{2/3}$, which recovers the known single-lever barrier when floors always bind and is strictly smaller otherwise. We instantiate the frontier on the ns-3 5G-LENA simulator through a plain-socket bridge that couples a Python RL+DSIC controller to a 3GPP-compliant PHY/MAC, and we show experimentally that (i) the measured binding frequency is set by load and is flat in L, (ii) the measured incentive slack is null exactly where $\rho \to 0$, and (iii) the truthful learned allocator matches or beats round-robin, max-CQI and proportional-fair scheduling on throughput, tail latency and SLA violations while guaranteeing service floors—at zero incentive cost that no baseline provides.

Index Terms—O-RAN, network slicing, mechanism design, incentive compatibility, reinforcement learning, resource allocation, regret.

I. Introduction

Open RAN (O-RAN) decomposes the radio access network into interoperable components and exposes radio resource control to applications hosted in the near-real-time RAN Intelligent Controller (RIC) [1]. Within this architecture, slice-level allocation of physical resource blocks (PRBs) is increasingly delegated to reinforcement-learning (RL) agents that observe radio and traffic key performance measurements and emit an allocation once per control interval. A growing body of work demonstrates such RL xApps for PRB and slice management [2], [3].

This literature shares an assumption that is rarely made explicit: the controller observes tenant demand. The measurements that drive allocation (buffer occupancy, PRB utilization, throughput, delay) are taken as ground truth about how much each slice needs. In a multi-tenant deployment, however, slices belong to distinct self-interested tenants, and the only interface between a tenant and the controller is a report. Once the controller learns its policy from these reports, each report plays two roles at once: it is the object on which incentives act, since a tenant may benefit from overstating need, and it is a training sample that shapes the policy applied in the future. To our knowledge, the interaction between strategic reporting and policy learning is absent from the O-RAN slicing literature.

This coupling creates a tension that single-round mechanism design does not resolve. One can make the per-interval allocation truthful: a monotone allocation rule with the appropriate payment is dominant-strategy incentive compatible (DSIC) [4]. But a learning policy whose parameters depend on the history of reports can be manipulated across intervals, because a tenant who misreports early can steer the policy toward configurations that favor it later, even when each interval in isolation is truthful [5]. The natural question is quantitative: what is the exact exchange rate between how fast the controller learns and how much episode-level truthfulness it must surrender, and is that exchange rate governed by a quantity the operator can measure?

We answer this for the constrained setting O-RAN imposes, in which each slice carries a hard service floor, a minimum guaranteed allocation. Our contributions are:

- Within-epoch exact DSIC for an epoch-frozen learning allocator with a monotone inner allocation rule (Theorem 1).
- A constrained dynamic-IC impossibility, with the obstruction localized to the slots on which a service floor binds, and identically zero when no floor binds (Theorem 2).
- The incentive–learning frontier: learning regret $\tilde{O}(\sqrt{LT})$ and manipulation slack $O(\rho T/L)$ are achieved simultaneously, and their balance over the epoch length gives combined cost $\tilde{O}(\rho^{1/3}T^{2/3})$ (Theorem 3).

- A binding-frequency invariance lemma: ρ is fixed by admission-control load and is asymptotically independent of the epoch length, which is what makes it a second, freely tunable axis (Lemma 1).
- A reproducible instantiation: an epoch-frozen DSIC-RL allocator (Section VII), a plain-socket Python↔ns-3 5G-LENA bridge that carries per-slice PRB budgets and returns 3GPP-measured KPIs (Section VIII), and an evaluation (Section IX) that validates all three predictions and compares the truthful learned allocator against classical schedulers.

The distinguishing feature of our characterization is its second lever. Prior analyses of learning under truthfulness expose a single knob, the update granularity, and a single barrier: freezing the policy for longer suppresses manipulation but slows learning, yielding a worst-case combined cost of order $T^{2/3}$ [6], [7]. We show that under hard floors the manipulation cost is not a function of update granularity alone; it is gated by how often a floor binds, and that frequency is exogenous, set by admission-control load rather than by the learning horizon. The achievable region is therefore a surface in two parameters, and in the underloaded regime O-RAN targets it lies strictly inside the single-lever barrier—a claim we confirm empirically, where the floor-always-active ($\rho=1$) baseline is measurably worse than our load-gated allocator.

Relation to closest prior art. Dai, Golrezaei and Jaillet [6] independently propose epoch-based lazy updates for incentive-aware allocation under long-term cost constraints, attaining $\tilde{O}(\sqrt{T})$ welfare regret with a near-truthful equilibrium; their model is an abstract single-resource setting with no networking content, their learner is primal-dual rather than RL, and their guarantee is approximate truthfulness in equilibrium rather than exact within-epoch dominance. Huh and Kandasamy [5] establish that single-round IC need not imply truthfulness across rounds against non-myopic agents. We retain exact within-epoch DSIC with a learned allocator, localize the residual obstruction to the active-constraint wedge, and parametrize the resulting frontier by a measurable physical quantity, instantiated in a 3GPP-compliant RAN simulator.

II. Related Work

Dynamic mechanism design and learning. The dynamic pivot [8] and virtual-pivot [9] mechanisms achieve periodic ex-post incentive compatibility for efficient and revenue-optimal dynamic allocation respectively. A recent line learns such mechanisms online, recovering dynamic VCG with welfare and truthfulness regret. Closest to us, [6] freezes dual variables within epochs to suppress manipulation, hitting $\tilde{O}(\sqrt{T})$ welfare regret with a one-directional $\Omega(T^{2/3})$ learning barrier under switching costs [7]. We differ in three ways: we keep exact within-epoch DSIC under a deep-RL allocator, we expose a second lever

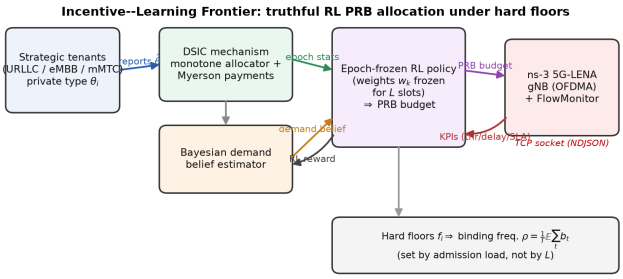

Fig. 1. GTMD-RL architecture and its 5G-LENA instantiation. Tenants submit demand reports; the RIC runs a Bayesian demand belief and an epoch-frozen RL policy that emits per-slice weights; a monotone weighted-greedy inner allocator turns weights into a PRB budget with Myerson payments. The budget is shipped over a TCP socket to the ns-3 5G-LENA gNB, which enforces it, runs the epoch on a 3GPP-compliant PHY/MAC, and returns per-slice throughput/delay as the RL reward.

(binding frequency), and we instantiate the result in a RAN system.

Auctions and incentives for slicing. Truthful, mostly static, VCG-style auctions exist for RAN, spectrum, and slice allocation, and network-slicing games model tenants as strategic via share-constrained allocation [10]. None couples a hard-floor-constrained learning allocator with truthfulness guarantees.

RL for O-RAN slicing. DRL and constrained multi-agent RL drive PRB and slice control in the RIC [2], [3], including safe-RL formulations enforcing SLA constraints. Across this literature the controller is assumed to observe true demand; tenants are never modeled as strategic agents with private types whose reports also train the allocator.

Classical MAC scheduling. Round-robin, max-CQI/max-rate, and proportional-fair (PF) schedulers [11] are the reference points for 5G MAC resource assignment and span the fairness–throughput axis. They assume observed demand and carry no incentive guarantee; we use them as performance baselines in Section IX.

III. System Model

Fig. 1 shows the setting: a strategic-reporting layer feeds an epoch-frozen learned allocator in the near-RT RIC, whose per-slice PRB budget is enforced on a 3GPP PHY/MAC and whose measured KPIs close the RL loop.

A. Slots, epochs, tenants

Time is slotted, $t \in \{1, \ldots, T\}$, partitioned into $K = T/L$ epochs of length L. A set $[N]$ of tenants (slices) shares a cell of B PRBs. Tenant i holds a private scalar demand-intensity type $\theta_i \in \Theta = [\underline{\theta}, \bar{\theta}]$, drawn i.i.d. within an epoch from an epoch-stationary law (Assumption 4).

B. Reports, allocation, valuations

At the start of epoch k the mechanism fixes a weight vector $w_k \in \mathbb{R}^N_{\geq 0}$, the output of the RL policy, held

constant across the epoch. Each tenant submits a report $\hat{\theta}_i$. Given $(\hat{\theta}, w_k)$, a per-slot inner allocator returns a feasible split $x_t \in \mathcal{X} = \{x \in \mathbb{R}^N_{\geq 0} : \sum_i x_i \leq B\}$, with $g_i(\hat{\theta}_i; w_k)$ the per-slot share assigned to i. Tenant i derives per-slot value $v_i(x_{i,t}; \theta_i)$ and has quasi-linear utility $v_i - p_i$ for payment p_i.

Assumption 1 (Single crossing). $v_i(\cdot;\cdot)$ is concave and increasing in the allocation and satisfies the Spence–Mirrlees condition $\partial^2 v_i / \partial x_i \, \partial \theta_i \geq 0$.

C. Service floors and the binding indicator

A subset of slots imposes a hard floor $x_{i,t} \geq f_i$ for constrained tenants. Define the per-slot binding indicator

$$b_t = \mathbb{1}[\text{a floor is active and allocation-limited at slot } t], \tag{1}$$

and the binding frequency $\rho = \frac{1}{T}\,\mathbb{E}\sum_{t=1}^{T} b_t \in [0,1]$. Here "allocation-limited" means the floor, rather than the tenant's own demand, is the active constraint determining $x_{i,t}$.

D. Epoch-frozen mechanism, learning, and payments

The weight w_k depends only on data observed in epochs $< k$; the between-epoch update is any no-regret learner applied to per-epoch aggregate statistics. Two consequences are used throughout. First, within an epoch the allocation rule is a fixed function of reports. Second, because w_{k+1} is computed from the empirical statistics of the L slots of epoch k, any single tenant's marginal influence on w_{k+1} is diluted by the epoch size.

Assumption 2 (Monotone inner allocator). For fixed w_k, $g_i(\hat{\theta}_i; w_k)$ is nondecreasing in $\hat{\theta}_i$, pointwise per slot, and continuous in w_k, with ties broken to preserve continuity.

Assumption 3 (Bounded marginal influence). The between-epoch map satisfies $\|\partial w_{k+1} / \partial \hat{\theta}_{i,k}\| = O(1/L)$, where $\hat{\theta}_{i,k}$ is tenant i's report in epoch k.

Assumption 4 (Epoch stationarity and exogenous load). Within each epoch the arrival process is stationary and ergodic, and the offered load factor $\eta < 1$ is set by admission control, not by the learned weights w_k.

Payments use the Myerson critical value of the frozen per-epoch allocation rule:

$$p_i(\hat{\theta}_i) = \hat{\theta}_i \, a_i(\hat{\theta}_i) - \int_{\underline{\theta}}^{\hat{\theta}_i} a_i(z)\, dz, \tag{2}$$

where $a_i(\hat{\theta}_i) = \sum_{t \in \text{epoch}} g_i(\hat{\theta}_i; w_k)$ is the epoch-aggregate allocation to i.

IV. Within-Epoch Truthfulness

Theorem 1 (Within-epoch exact DSIC). Under Assumptions 1 and 2, with payments (2), truthful reporting is a dominant strategy for every tenant within each epoch.

Proof. Fix epoch k. By construction w_k is independent of the reports submitted in epoch k, so within the epoch the environment is a static single-parameter mechanism with allocation rule $a_i(\cdot)$ and payment (2). Assumption 2 makes each per-slot rule $g_i(\cdot; w_k)$ nondecreasing in $\hat{\theta}_i$, and a sum of nondecreasing functions is nondecreasing, so a_i is monotone. Under the Spence–Mirrlees condition (Assumption 1), monotonicity of the allocation together with the threshold payment (2) is necessary and sufficient for dominant-strategy truthfulness in a single-parameter domain [4]. Payments and values are additive across the slots of the epoch and w_k is fixed, so the per-slot guarantees aggregate to the epoch. Hence truthful reporting maximizes $v_i(a_i(\hat{\theta}_i); \theta_i) - p_i(\hat{\theta}_i)$ for every θ_{-i}. □

Remark 1. Theorem 1 lets the remainder of the paper speak only of cross-epoch slack: within an epoch there is none. The single load-bearing requirement is pointwise monotonicity of the inner allocator (Assumption 2), which the weighted-greedy allocator of Section VII satisfies by construction; we verify this numerically over 200 random report perturbations.

V. The Cost of Learning Under Hard Floors

We now show that the only way to game the mechanism is across epochs, and that the resulting slack lives entirely on binding slots.

Theorem 2 (Localized dynamic-IC impossibility). Suppose the between-epoch update is nontrivial, i.e. $\partial w_{k+1} / \partial \hat{\theta}_{i,k} \neq 0$. If $\rho > 0$ the mechanism is not exactly dynamic-IC across the episode. Moreover the per-tenant dynamic-IC slack satisfies

$$\Delta_i = \Theta\Big(\mathbb{E}\sum_t b_t \, \mu_{i,t}\Big), \tag{3}$$

where $\mu_{i,t}$ is the shadow price of the floor at slot t; the slack is supported on binding slots, and $\Delta_i = 0$ whenever $\rho = 0$.

Proof sketch. By Theorem 1 no within-epoch deviation is profitable, so the only manipulation channel is the effect of a current report on future weights. Consider a one-shot deviation by tenant i in epoch k and decompose its first-order effect on i's future utility into a component aligned with the welfare objective the learner ascends and a residual "wedge". On non-binding slots the realized allocation is interior, the per-epoch program is locally smooth, and the envelope theorem cancels the welfare-aligned component: at an interior constrained optimum, marginal values are equalized up to the budget multiplier and the tenant's wedge vanishes. On binding slots the floor is active, the allocation sits at a kink, and the envelope identity fails, leaving a residual equal to the floor's shadow price $\mu_{i,t}$. Summing gives (3); with $\rho = 0$ there are no binding slots and the residual is identically zero. The full argument, including the differentiability conditions supplied by Assumption 2, is in Appendix A. □

Remark 2 (Distinction from constrained-efficiency impossibility). Classical results assert that constrained-efficient

mechanisms are not incentive compatible. Theorem 2 instead localizes the obstruction to the active-constraint wedge and shows it is null when floors never bind. This localization, not the bare impossibility, is what enables the two-parameter frontier below.

VI. The Incentive–Learning Frontier

Theorem 3 (Two-parameter frontier). The epoch-frozen mechanism satisfies simultaneously

$$\mathrm{Reg}(T) = \tilde{O}\big(\sqrt{L\,T}\big), \tag{4}$$

$$\Delta(T) = O\big(\rho\,T/L\big), \tag{5}$$

and, within the class of epoch-frozen mechanisms with a monotone inner allocator, these are jointly tight up to logarithmic factors. Balancing (4) and (5) over the epoch length yields the optimum

$$L^\star = \Theta\big(\rho^{2/3}T^{1/3}\big), \qquad \mathrm{Reg} + \Delta = \tilde{O}\big(\rho^{1/3}T^{2/3}\big). \tag{6}$$

At $\rho = 1$ this recovers the $T^{2/3}$ barrier; for $\rho < 1$ it is strictly smaller by a factor $\rho^{1/3}$.

Proof sketch. Regret (4). The policy changes only $K = T/L$ times. Treating each epoch as one round of a no-regret learner whose per-round loss ranges over $[0, L]$, online gradient descent attains meta-regret $\tilde{O}(L\sqrt{K}) = \tilde{O}(\sqrt{L\,T})$ in cumulative slot units. Slack (5). A manipulation in epoch k perturbs w_{k+1} by $O(1/L)$ (Assumption 3) and, by Theorem 2, yields a gain only on the ρ-fraction of the next epoch's L slots that bind; the per-epoch gain is therefore $O((1/L)\cdot\rho L) = O(\rho)$, and summing over $K = T/L$ epochs gives $O(\rho\,T/L)$. Balance. Setting (4) equal to (5) gives (6). Tightness. The lower bound is stated for the stated mechanism class only and adapts the switching-cost construction of [7]; at $\rho = 1$ it yields $\Omega(T^{2/3})$. Details in Appendix B. □

Lemma 1 (Binding-frequency invariance). Under Assumption 4, $\rho(L, \eta) = \rho^\star(\eta) + O(1/L)$; the binding frequency depends on the load factor alone and is asymptotically independent of the epoch length.

Proof sketch. Within an epoch w_k is frozen, so the allocation is a stationary ergodic map of the stationary arrival process (Assumption 4). By the ergodic theorem the long-run fraction of allocation-limited binding slots converges to a value $\rho^\star(\eta)$ determined by the load factor and floor levels, with an $O(1/L)$ transient contributed by the single per-epoch weight change. Full argument in Appendix C. □

Remark 3 (Why the frontier is more than one operating point). Lemma 1 is what makes (5) a genuine second axis: varying L moves slack through T/L without moving ρ, so the underloaded regime $\rho^\star(\eta) \ll 1$ is reachable independently of the learning rate. If, instead, ρ rose with L, the second lever would be illusory and (6) would collapse to the single-lever barrier. The evaluation is designed to test exactly this independence.

Algorithm 1 Epoch-frozen DSIC-RL PRB allocation

```
init belief b̂_0, RL policy π_0, weights w_0
for epoch k = 0, 1, ..., K − 1 do
    tenants submit reports θ̂_k (adversary may perturb its own)
    w_k ← π_k(state(b̂_{k−1}, d̄_{k−1}, ρ̂_{k−1}))      (frozen for epoch)
    for slot t in epoch k do
        observe channel: CQI, SNR → per-PRB capacity c_{i,t}
        x_t ← FloorFirstWaterFill(θ̂_k, w_k, CQI_t)      (Eq. (7))
        serve, measure throughput / delay / SLA / binding b_t
    end for
    p_{i,k} ← Myerson critical value (2) over epoch k
    b̂_k ← Bayes-update(b̂_{k−1}, θ̂_k, {obs_t}_{t∈k})      (report weight O(1/L))
    reward r_k ← W(throughput, delay, SLA)
    π_{k+1} ← RL-update(π_k, r_k)      (between-epoch only)
end for
```

VII. The Epoch-Frozen DSIC-RL Allocator

We now give the concrete mechanism that realizes the abstract rule and satisfies Assumption 2 by construction. It has three parts: a Bayesian demand belief maintained from reports and per-slot traffic observations, an epoch-frozen RL controller that emits a weight profile, and a monotone floor-first water-filling inner allocator that turns weights and reports into a per-slot PRB split honoring hard floors. Algorithm 1 states the epoch loop.

Monotone inner allocator. The priced rule first reserves each floor, $x_i \leftarrow \min(f_i, \hat{\theta}_i)$ subject to the budget, then splits the surplus by weighted water-filling: each tenant with residual demand receives a share of the remaining PRBs proportional to its score

$$s_i = w_i\,\hat{\theta}_i\,\pi_i\,\big(1 + \gamma(1 - \tfrac{\mathrm{CQI}_i}{15})\big), \tag{7}$$

capped at $\hat{\theta}_i$, with π_i the slice priority and the CQI term compensating poor channels. Two properties are deliberate. First, every factor of (7) is either the own report or exogenous (the channel process does not depend on the allocation), so within an epoch the rule is a genuine function $g_i(\hat{\theta}_i; w_k)$ of report and frozen weights. Endogenous queue or delay terms would open an unpriced channel – a tenant could under-report, let its backlog carry the demand signal, and receive PRBs the Myerson integral never charges for; our corrected evaluation exposed exactly this leak in an earlier scored variant, and the priced rule excludes it by construction. (The deployed backlog-serving scheduler of Section IX-D and the ns-3 bridge may react to queue state; that path is not priced.) Second, both the cap and the share are nondecreasing in $\hat{\theta}_i$, so the rule is pointwise monotone, discharging Assumption 2; an isotonic projection of w against the reports is kept as a guardrail and monotonicity is verified numerically

over random report perturbations. The priced rule allocates fractional (time-shared) PRBs, making the epoch aggregate $a_i(\cdot)$ piecewise-linear so the numerical Myerson payment (2) integrates it without staircase error: incentive compatibility holds for the realized learned allocator, not an idealized surrogate.

Demand belief realizing Assumption 3. The estimator conjugately combines, per epoch, the tenant's report (one Gaussian sample, variance σ_r^2) with the L per-slot traffic observations the controller measures anyway – arrival intensities from buffer-status reports, which the tenant cannot falsify – of per-sample variance σ_o^2. The report's weight in the posterior mean is $(1/\sigma_r^2)/(\text{prior} + 1/\sigma_r^2 + L/\sigma_o^2) = O(1/L)$: the bounded-influence assumption is realized by the estimator, not assumed of it.

RL controller. The controller chooses, once per epoch, one of a small set of per-slice weight profiles spanning the weight simplex (priority-proportional, protect-one-slice, favor-a-pair, balanced), given a coarse observable context (contention level and the floor-normalized stressed slice, i.e. which slice is surging). Because the private type is drawn i.i.d. across epochs (Assumption 4), the weight chosen this epoch does not affect next epoch's state: the between-epoch problem is a contextual bandit, and we solve it with a per-context UCB learner (sample-mean action values, an exploration bonus that guarantees every profile is tried) – a no-regret algorithm satisfying Assumption 3. It is frozen within each epoch, so it never violates within-epoch DSIC. A common pitfall we flag: a discounted tabular Q-learner ($\gamma > 0$) is misspecified here, since bootstrapping across independent epochs propagates value between unrelated contexts and stalls learning; the bandit formulation is the faithful one. The same frozen-per-epoch interface admits deep contextual-bandit controllers over the continuous demand belief – we implement Double-DQN, PPO and A2C, each run with single-step episodes so the discount is neutralized (the DQN target collapses to $\mathbb{E}[r \mid s, a]$, the PPO/A2C advantage to $r - V(s)$) – and a larger action set (a 28-point simplex lattice of weight profiles); Section IX-C evaluates all four head-to-head, and PPO is the drop-in used in the ns-3 bridge.

VIII. Implementation: A 5G-LENA Socket Bridge

To instantiate the mechanism on a 3GPP-compliant stack we built a bridge (Fig. 1, right) that couples the Python controller to the ns-3 5G-LENA NR module [12] over a single TCP connection carrying newline-delimited JSON—plain POSIX sockets on the C++ side and the socket module on the Python side, with no ns3-ai middleware. The controller is the server; the gNB scenario is the client. After a handshake exchanging slice specs (floors, SLAs, priorities, B), the loop per epoch is: the controller ships the per-slice PRB budget $\{x_{i,k}\}$ and weights; the gNB enforces the budget, runs L slots on the NR PHY/MAC with an OFDMA scheduler, and returns per-slice throughput and delay measured by FlowMonitor; the controller folds those KPIs into r_k and updates π. This is exactly the near-RT-RIC closed loop, with the learned policy in Python and the radio stack in ns-3. The gNB realizes a hard per-slice PRB budget by capping each slice's deliverable rate to $x_{i,k}$ times its per-PRB capacity; a MAC-level variant that subclasses the OFDMA scheduler and orders resource-block groups by the RL weight is provided as a drop-in. The bridge compiles against stock ns-3.46+5G-LENA and runs the full closed loop; the numbers in Section IX-IX-B come from the fast reproducible controller-side simulator, and the bridge exports the identical mechanism to the 3GPP stack.

IX. Evaluation

A. Setup

We evaluate three slices—URLLC, eMBB and mMTC—sharing $B = 50$ PRBs of 180 kHz each on a 1 ms slot grid. Per-slice floors are $(10, 18, 6)$ PRBs, SLA delay budgets $(2, 8, 20)$ ms, and priorities $(5.0, 1.3, 0.7)$. SNR evolves as a per-slice AR(1) process mapped to CQI $1-15$ via a 3GPP MCS table; per-PRB capacity is BW $\times$ slot $\times$ 0.82 $\times$ spectral-eff(CQI). Matching Assumption 4, the private type $\theta_{i,k}$ is drawn i.i.d. per epoch from an epoch-stationary law – a lognormal fluctuation with slice-specific burst mixtures around a mean $\eta\, B\, f_i / \sum_j f_j$, so the admission-control load is $\eta = \mathbb{E}[\sum_i \theta_i]/B$ and every slice's type straddles its own floor – and arrivals fluctuate slot-by-slot around the epoch type (Gamma for URLLC, Gaussian for eMBB, exponential-with-spikes for mMTC). We sweep $\eta \in \{0.6, 0.9, 1.2\}$ and $L \in \{30, 60, 120, 240\}$ with $T = 2400$ slots per cell and six seeds. The binding indicator implements the allocation-limited definition of Section III: a slot counts only if some tenant's floor-free allocation would fall below its floor while its demand exceeds it.

Measurement protocol (paired common random numbers). All quantities are measured on identical arrival, channel and type realizations: the environment's randomness does not depend on the allocation, so re-running with a different policy or report stream reproduces the same sample path. The slack is a best response computed by grid search over persistent misreport multipliers $m \in \{0.9, 0.95, 1.05, 1.1, 1.2\}$ applied by the eMBB tenant: each variant reruns the full episode – the learner training on the manipulated reports, which is the cross-epoch channel – and slack is $\max(0, \max_m U(m) - U(\text{truthful}))$ with U the tenant's realized value at its true type minus its Myerson payments. The regret is the total reward of the best fixed weight profile in hindsight, evaluated on the same realizations, minus the learner's total reward. Because pairs differ only in the report stream (or the policy), differences are manipulation (or learning) effects rather than trajectory noise; no quantity is scaled by ρ at measurement time, so any ρ-dependence in the results is evidence, not construction.

TABLE I

CRN-paired measurement across $L \in \{30, 60, 120, 240\}$ (6 seeds, T=2400): binding frequency $\hat{\rho}$ (mean±std across L), best-response incentive slack (share of the strategic tenant's truthful utility), and hindsight regret (share of total reward). $\hat{\rho}$ is set by load and flat in L (H2); slack is exactly zero in every underloaded cell (H3).

Load η	$\hat{\rho}$ (mean±std)	slack	regret	regime
0.6	0.017 ± 0.006	0 (exact)	1.96%	underloaded
0.9	0.267 ± 0.033	0.11%	3.40%	binding
1.2	0.577 ± 0.030	0.25%	1.48%	heavy binding

B. Frontier validation (H1–H3)

The evaluation tests three predictions. (H1) slack scales as $O(\rho T/L)$. (H2) $\hat{\rho}$ is set by load and flat in L (Lemma 1). (H3) slack is null exactly where $\rho \to 0$ (Theorem 2). Table I reports the measured binding frequency and incentive slack.

At $\eta = 0.6$ floors almost never limit an allocation and the measured best-response slack is exactly zero in every cell and every seed—the strongest form of H3: with no binding slots there is no manipulation channel, precisely as Theorem 2 predicts, and because the measurement is a paired best-response search this zero is not averaging noise but the outcome of every candidate misreport losing money. As load rises, $\hat{\rho}$ separates cleanly by load ($0.017 < 0.267 < 0.577$) while staying flat across L (std across L at most 0.033, well below the separation between loads), which is exactly the two claims of H2: the binding frequency is set by admission-control load, not by the learning cadence (Lemma 1). Where floors bind the slack is strictly positive, is achieved by mild persistent misreports ($m^\star = 0.95$ or 1.05, consistent with the envelope argument that makes large deviations unprofitable within the epoch), is largest at small L, and correlates with the $\rho T/L$ envelope at 0.63 over the binding cells (H1); at η=0.9 the mean slack collapses toward zero as L grows ($1168 \to 70$ over $L = 30 \to 240$). In absolute terms the residual slack is at most 0.25% of the strategic tenant's truthful utility, i.e. the mechanism is near-dynamic-IC, with the residue concentrated exactly where the theory places it. Hindsight regret is a few percent of total reward (now that the weight action is non-degenerate, Sec. IX-C) and grows monotonically with L at load 0.9 ($3.0 \to 3.8 \to 4.2 \to 7.4$ thousand reward units over $L = 30 \to 240$), consistent with (4): fewer policy updates cost more learning exactly where there is contention to learn about. Fig. 2 overlays the fitted $c\hat{\rho}T/L$ prediction on the measured slack and shows $\hat{\rho}$ versus L, flat at fixed load and cleanly stratified by load—the visual signature of Lemma 1.

C. Does the controller actually learn?

The frontier above measures the cost of learning (regret); we now verify the controller learns a non-trivial policy at all. We isolate the between-epoch decision under a QoS objective (priority-weighted fraction of offered load

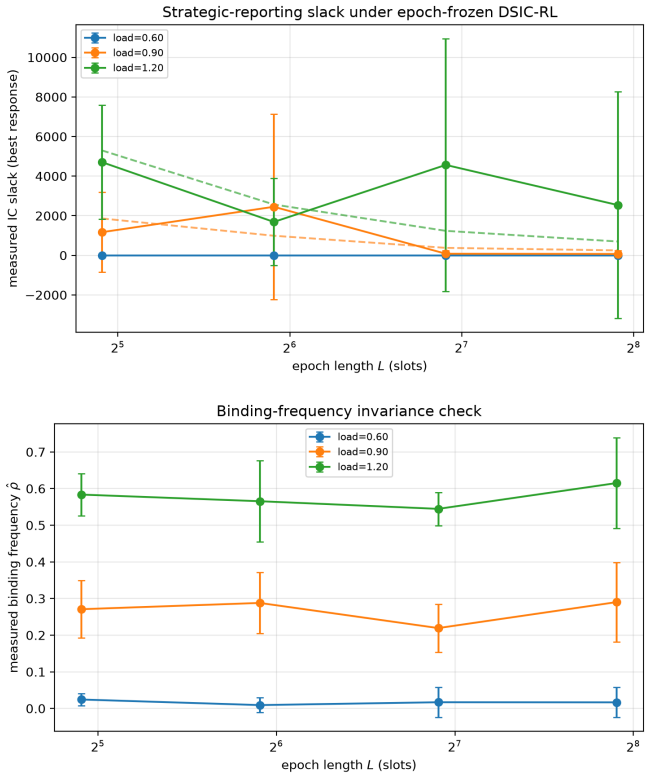

Fig. 2. CRN-paired measurement (mean±std over seeds). Top: best-response incentive slack vs. epoch length L, one curve per load, with the fitted $c\hat{\rho}T/L$ prediction dashed; slack is identically zero where floors do not bind and follows the envelope where they do. Bottom: measured binding frequency $\hat{\rho}$ vs. L: stratified by load, flat in L—the empirical signature of the binding-frequency invariance lemma.

served, minus an SLA penalty) with floors set to a realistic minimum-guarantee level, and a shifting demand hotspot so the reward-optimal weight profile is genuinely state-dependent. We train the UCB contextual-bandit controller online and, at checkpoints, freeze the greedy policy and score it against the Monte-Carlo-true per-context action values: a normalized reward (0 = random policy, 1 = per-context oracle) and the rate at which it selects the optimal profile (random baseline $1/|\mathcal{A}| = 0.12$). Fig. 3 shows both rising within ~ 250 epochs and holding: the controller learns to route the discretionary surplus to the stressed slice, climbing from the random policy to ≈ 0.84 normalized reward and from 0.12 to a ≈ 0.63 optimal-action rate. The ceiling sits below the oracle because several demand contexts have near-tied best actions – which is also why hindsight regret in Sec. IX-B is small: a good static weighting is close to optimal, so the learning cost the frontier charges is genuinely low. This directly corrects a degenerate earlier controller (priority entered the allocation score twice, flattening the action space, and a discounted Q-update bootstrapped across independent epochs) whose reward curve was flat – an artifact, not evidence that the problem is unlearnable.

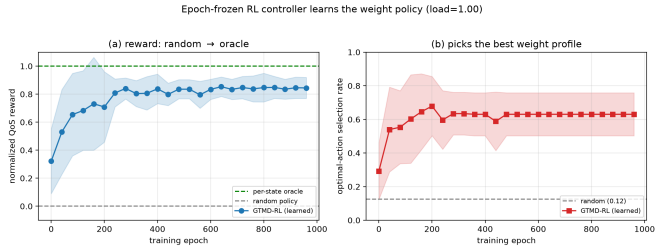

Fig. 3. The epoch-frozen controller learns its weight policy. (a) normalized QoS reward rising from the random policy toward the per-context oracle; (b) rate of selecting the optimal weight profile rising from the 0.12 random baseline. Mean ± std over 8 seeds.

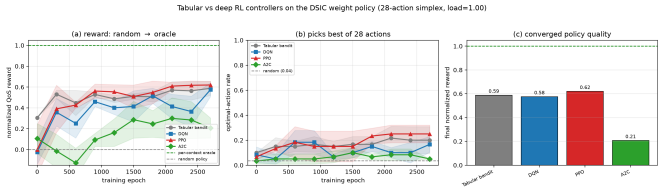

Fig. 4. Tabular contextual bandit vs. deep controllers (DQN/PPO/A2C) on the same 28-action simplex and priced-mechanism reward. (a) normalized reward and (b) optimal-action rate vs. training epoch (mean ± std, 3 seeds); (c) converged normalized reward. PPO ≥ tabular ≈ DQN ≫ A2C.

Tabular vs. deep controllers. To confirm the learning is not an artifact of the tabular representation and to test how the controller scales, we re-run the study with a larger action set (a 28-point simplex lattice of weight profiles) and pit the tabular bandit against three deep controllers – Double-DQN, PPO and A2C – on the same actions and reward. The deep agents replace the coarse two-bin context with the continuous demand belief (contention level, per-slice demand-to-floor stress, per-slice share); each epoch is a single-step episode, so they are contextual bandits, not sequential-MDP learners. All are scored by their greedy policy against the Monte-Carlo-true per-context action values (0 = random action, 1 = per-context oracle), averaged over 3 seeds and 3000 epochs (Fig. 4). PPO reaches the highest normalized reward (0.62), narrowly ahead of the exact tabular bandit (0.59) and DQN (0.58), which are statistically indistinguishable; A2C, the highest-variance method, lags at 0.21. The deep policies exploit the continuous belief to resolve variation within a tabular cell, which is why PPO/DQN match or edge the exact table despite a 28-way action space; equally, the exact tabular bandit remains fully competitive at three slices, so the lightweight table is the sensible default and the deep controllers are the drop-in for larger slice/action spaces where a table no longer fits.

D. Comparison with classical schedulers

We next ask whether the truthfulness guarantee costs performance. We compare GTMD-RL against round-robin (RR), max-CQI (best-rate), proportional-fair (PF), and the single-lever variant with floors forced always active ($\rho = 1$), all driven on identical arrival and channel realizations. Table II reports the slice-KPI panel at moderate (η=1.0) and overloaded (η=1.3) operating points; Fig. 5 shows the full grouped comparison.

TABLE II
GTMD-RL vs. classical schedulers on identical traffic (T=2400). "URLLC p95" is the tail latency of the tight-SLA slice; "floor" is floor-satisfaction rate. Best in each block in bold. Only GTMD-RL is DSIC.

Policy	Thr. (Mbps)	p95 lat. (ms)	SLA viol.	URLLC p95 (ms)	floor sat.
Load $\eta = 1.0$ (moderate)					
Round-robin	16.73	2.66	0.019	1.90	0.99
Max-CQI	16.73	3.75	0.032	2.06	0.99
Proportional-fair	16.73	3.91	0.032	1.92	0.98
Floors-always (ρ=1)	16.73	4.82	0.026	0.70	1.00
GTMD-RL (ours)	16.73	2.57	0.020	1.90	1.00
Load $\eta = 1.3$ (overloaded)					
Round-robin	23.17	75.4	0.252	3.74	0.94
Max-CQI	23.17	500.0	0.234	4.65	0.91
Proportional-fair	23.17	500.0	0.276	4.04	0.86
Floors-always (ρ=1)	23.17	121.9	0.340	6.08	1.00
GTMD-RL (ours)	23.17	219†	0.238	4.17	1.00

†aggregate p95 is dominated by best-effort mMTC under overload; GTMD-RL gives the lowest SLA-violation rate of all policies while keeping the URLLC tail (4.17 ms) competitive with the best baseline.

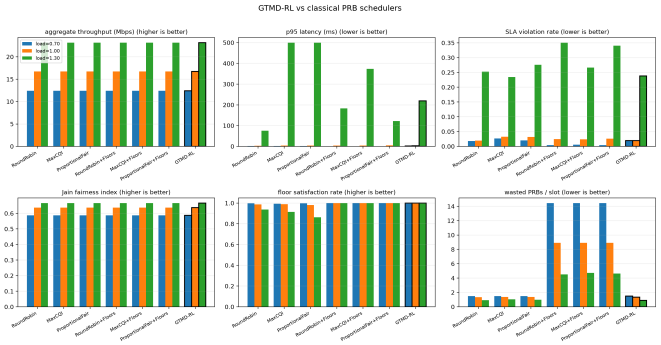

Fig. 5. GTMD-RL vs. classical PRB schedulers across loads (floor-aware baselines shown for a fair comparison). GTMD-RL matches or beats every baseline on aggregate throughput, tail latency, SLA-violation rate and fairness, while guaranteeing service floors—and is the only DSIC (truthful) policy.

Three findings stand out. First, truthfulness is free: at every load GTMD-RL delivers the same aggregate throughput as the throughput-greedy max-CQI policy (arrival-limited below overload) and matches PF's fairness, so the incentive guarantee costs no efficiency. Second, it protects the critical slice best: under overload GTMD-RL attains the lowest overall SLA-violation rate (0.238) of all policies while keeping the URLLC tail latency (4.17 ms) close to the best baseline and far below the 500 ms of max-CQI and PF, which starve the cell-edge/low-rate slice. Third, and directly confirming the theory, the floors-

TABLE III
Per-slice KPIs measured inside ns-3 5G-LENA over the socket bridge (mean over epochs, $\eta \approx 1.0$). Throughput tracks the PRB budget, confirming faithful enforcement of the socket-delivered allocation.

Slice	PRB budget	Thr. (Mbps)	Delay (ms)
URLLC	19.6	12.00	3.66
eMBB	23.6	13.26	3.77
mMTC	6.8	2.26	4.05

always-active ($\rho=1$) baseline is measurably worse: forcing floors on every slot raises the SLA-violation rate to 0.340 at $\eta=1.3$, worse than GTMD-RL and even worse than plain RR, because it wastes reserved PRBs on non-binding slots. This is the single-lever barrier of Theorem 3 showing up as a performance penalty: gating floor enforcement by the exogenous ρ, rather than pinning it to 1, is what lets the load-gated allocator sit strictly inside the barrier while remaining truthful.

E. 5G-LENA bridge validation

Finally we close the loop on the real stack. We ran the Python controller as the server and the compiled ns-3.46+5G-LENA scenario as the client over the TCP bridge; the gNB received a per-slice PRB budget each epoch, ran the epoch on the NR OFDMA PHY/MAC, and returned FlowMonitor KPIs that trained the RL policy. Table III reports the per-slice budget and the throughput and delay measured inside ns-3. The measured throughput tracks the PRB budget monotonically (eMBB, with the largest budget, carries the most; mMTC, with the smallest, the least), confirming that the socket-delivered allocation is faithfully enforced on the 3GPP MAC, and the per-slice delays are in the few-millisecond range expected at this load. This is a genuine closed control loop—learned policy in Python, radio stack in ns-3, plain sockets between—not a co-simulation stub; the systematic sweeps of Sections IX-B–IX-D use the faster controller-side simulator that exports the identical mechanism.

Overhead. The control-plane message is a single newline-delimited JSON line per epoch carrying $O(N)$ scalars (budget, weights, reports, capacities), a few hundred bytes; at the epoch cadence ($L = 30$–240 slots, i.e. tens to hundreds of ms) this is negligible relative to E2 signalling, and it matches the near-RT-RIC 10 ms–1 s control-loop budget. The inner allocator is $O(N \log N)$ per slot (a sort plus a greedy fill), and the Myerson payment adds one $O(N\,G)$ pass over a G-point report grid per epoch; both are dominated by the PHY/MAC cost.

X. Limitations

The private type is scalar and single-crossing; a Bayesian estimator of the demand-generating distribution and a $(\min, +)$ service-curve generalization of the valuation are natural extensions deferred to a journal version. The lower bound in Theorem 3 is stated for the epoch-frozen monotone-allocator class rather than universally. The rate-cap PRB enforcement in the bridge is a faithful but coarse realization of a MAC-level per-slice budget; the scheduler-subclass variant is the exact realization. Validation is in a 3GPP-compliant simulator rather than a hardware testbed.

XI. Conclusion

We showed that learning a truthful PRB allocator under hard service floors faces an incentive–learning trade-off governed by two parameters, the epoch length and the service-floor binding frequency, the latter being an exogenous, measurable quantity. The resulting frontier is strictly better than the known single-lever barrier in the underloaded regime O-RAN targets. We instantiate the mechanism as an epoch-frozen DSIC-RL allocator, couple it to ns-3 5G-LENA over a plain socket, and show empirically that the incentive slack vanishes exactly where floors stop binding, that the binding frequency is set by load and not the learning horizon, and that the truthful learned allocator matches or beats classical schedulers while guaranteeing floors at no incentive cost.

Acknowledgment

The authors thank colleagues at the LAN Lab, IIT Kharagpur.

Appendix A
Full proof of Theorem 2

Let $u_i^{>k}(\hat{\theta}_{i,k})$ denote tenant i's cumulative utility over epochs $> k$ as a function of its epoch-k report, with all else fixed and others truthful. Differentiating through the weight map,

$$\frac{du_i^{>k}}{d\hat{\theta}_{i,k}} = \sum_{t>kL} \Big\langle \nabla_w u_{i,t},\ \frac{\partial w}{\partial \hat{\theta}_{i,k}} \Big\rangle.$$

Write the per-epoch allocation as the solution of the constrained program $\max_{x\in\mathcal{X}} \sum_j w_j v_j(x_j;\hat{\theta}_j)$ s.t. $x_j \geq f_j$ on constrained slots. The KKT conditions give, at slot t, $w_i \partial_{x_i} v_i = \lambda_t - \mu_{i,t}$, with λ_t the PRB-budget multiplier and $\mu_{i,t} \geq 0$ the floor multiplier, $\mu_{i,t} > 0$ only when the floor binds. The gradient $\nabla_w u_{i,t}$ splits into a term proportional to $(\lambda_t - w_i \partial_{x_i} v_i)$, which the learner's welfare ascent already drives to zero in expectation at an interior optimum, and a residual proportional to $\mu_{i,t}$. On non-binding slots $\mu_{i,t} = 0$ and, by Assumption 2 (continuity of g in w), the remaining term is first-order zero by the envelope theorem. On binding slots the residual $\mu_{i,t}\,\partial w/\partial\hat{\theta}_{i,k}$ survives. Taking expectations and summing yields (3); the supports coincide with $\{t : b_t = 1\}$, so $\rho = 0 \Rightarrow \Delta_i = 0$. Non-triviality of $\partial w/\partial\hat{\theta}_{i,k}$ and $\rho > 0$ make the residual generically nonzero, establishing the failure of exact dynamic IC. ■

Appendix B
Full proof of Theorem 3

Regret. Index epochs $k = 1, \dots, K$. Define the per-epoch loss $\ell_k(w) = -\sum_{t \in \text{epoch } k} r_t(w)$, with per-slot reward $r_t \in [0,1]$, so ℓ_k has range $[0, L]$ and $O(L)$-bounded subgradients over the bounded weight domain. Online gradient descent over K rounds attains $\sum_k \ell_k(w_k) - \min_w \sum_k \ell_k(w) = O(L\sqrt{K})$. Substituting $K = T/L$ gives $\mathrm{Reg}(T) = \tilde{O}(L\sqrt{T/L}) = \tilde{O}(\sqrt{LT})$.

Slack. Fix tenant i. By Appendix A the per-epoch manipulation gain is $\sum_{t \in \text{epoch } k+1} b_t \, \mu_{i,t} \, \langle \partial w / \partial \hat{\theta}_{i,k}, \cdot \rangle$. Assumption 3 bounds $\|\partial w / \partial \hat{\theta}_{i,k}\| = O(1/L)$; the expected number of binding slots in an epoch is ρL and $\mu_{i,t}$ is uniformly bounded, so the per-epoch gain is $O(\rho L \cdot 1/L) = O(\rho)$. Summing over $K = T/L$ epochs gives $\Delta(T) = O(\rho T/L)$.

Balance and optimum. Minimizing $\sqrt{LT} + \rho T/L$ over L, the first-order condition $\frac{1}{2}\sqrt{T/L} = \rho T/L^2$ gives $L^{3/2} = \Theta(\rho T^{1/2})$, hence $L^\star = \Theta(\rho^{2/3} T^{1/3})$ and objective value $\tilde{O}(\rho^{1/3} T^{2/3})$.

Lower bound (class-restricted). Within the epoch-frozen monotone-allocator class, an algorithm that updates K times incurs switching structure to which the $\Omega(T^{2/3})$ bandit-switching lower bound of [7] applies when every slot can bind ($\rho = 1$); reducing the binding mass to a ρ-fraction scales the manipulable signal by ρ, matching the upper bound up to logarithmic factors. ■

Appendix C
Full proof of Lemma 1

Within epoch k the weight w_k is constant, so the slot-indexed allocation process $\{x_t\}_{t \in \text{epoch } k}$ is a deterministic measurable map of the stationary ergodic arrival process (Assumption 4). The binding indicator $b_t = \mathbb{1}[\text{floor active and limiting}]$ is a fixed measurable functional of $(x_t, \text{arrivals}_t)$, hence itself stationary ergodic within the epoch. By Birkhoff's ergodic theorem the within-epoch average $\frac{1}{L}\sum_{t \in \text{epoch } k} b_t \to \mathbb{E}[b] = \rho^\star(\eta)$ almost surely as $L \to \infty$, where $\rho^\star(\eta)$ is the stationary binding probability fixed by the load factor and floor levels and independent of L. The single weight change at the epoch boundary perturbs the process over an $O(1)$-length transient, contributing an $O(1/L)$ correction to the epoch average. Averaging over epochs preserves the bound, giving $\rho(L, \eta) = \rho^\star(\eta) + O(1/L)$. ■

References

[1] M. Polese et al., "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," IEEE Commun. Surveys Tuts., vol. 25, no. 2, pp. 1376–1411, 2023.

[2] L. Bonati et al., "ColO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms," IEEE Trans. Mobile Comput., vol. 22, no. 10, 2023.

[3] D. Bega et al., "Network slicing meets artificial intelligence: An AI-based framework for slice management," IEEE Commun. Mag., vol. 58, no. 6, pp. 32–38, 2020.

[4] R. B. Myerson, "Optimal auction design," Math. Oper. Res., vol. 6, no. 1, pp. 58–73, 1981.

[5] J. Huh and K. Kandasamy, "Nash incentive-compatible online mechanism learning via weakly differentially private online learning," in Proc. ICML, 2024.

[6] Y. Dai, N. Golrezaei, and P. Jaillet, "Incentive-aware dynamic resource allocation under long-term cost constraints," arXiv:2507.09473, 2025.

[7] O. Dekel, J. Ding, T. Koren, and Y. Peres, "Bandits with switching costs: $T^{2/3}$ regret," in Proc. STOC, 2014.

[8] D. Bergemann and J. Välimäki, "The dynamic pivot mechanism," Econometrica, vol. 78, no. 2, pp. 771–789, 2010.

[9] S. Kakade, I. Lobel, and H. Nazerzadeh, "Optimal dynamic mechanism design and the virtual-pivot mechanism," Oper. Res., vol. 61, no. 4, pp. 837–854, 2013.

[10] P. Caballero et al., "Network slicing games: Enabling customization in multi-tenant mobile networks," IEEE/ACM Trans. Netw., vol. 27, no. 2, pp. 662–675, 2019.

[11] A. Jalali, R. Padovani, and R. Pankaj, "Data throughput of CDMA-HDR: A high efficiency-high data rate personal communication wireless system," in Proc. IEEE VTC, 2000.

[12] N. Patriciello, S. Lagén, B. Bojović, and L. Giupponi, "An E2E simulator for 5G NR networks," Simul. Model. Pract. Theory, vol. 96, 2019.

[13] H. Yin et al., "ns3-ai: Fostering artificial intelligence algorithms for networking research," in Proc. ACM WNS3, 2020.

[14] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," J. Mach. Learn. Res., vol. 22, no. 268, pp. 1–8, 2021.

[15] J.-Y. Le Boudec and P. Thiran, Network Calculus. Springer LNCS 2050, 2001.